

Entrega 4. Comparativa SQLJ frente a JDBC, buenas prácticas, ejercicios y resumen final

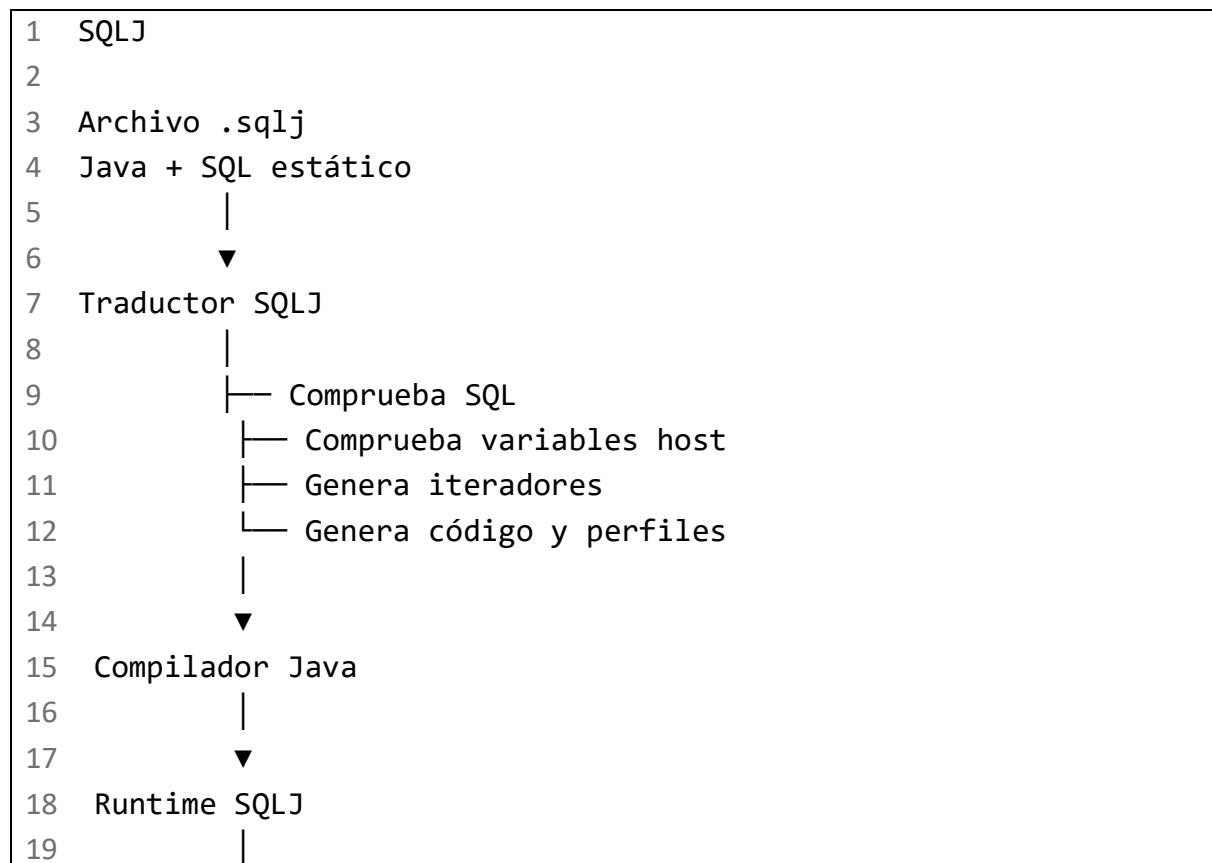
Esta última entrega cierra la guía con una comparación práctica entre SQLJ y JDBC, criterios de elección, buenas prácticas, un ejercicio integral resuelto y ejercicios adicionales desde nivel básico hasta avanzado.

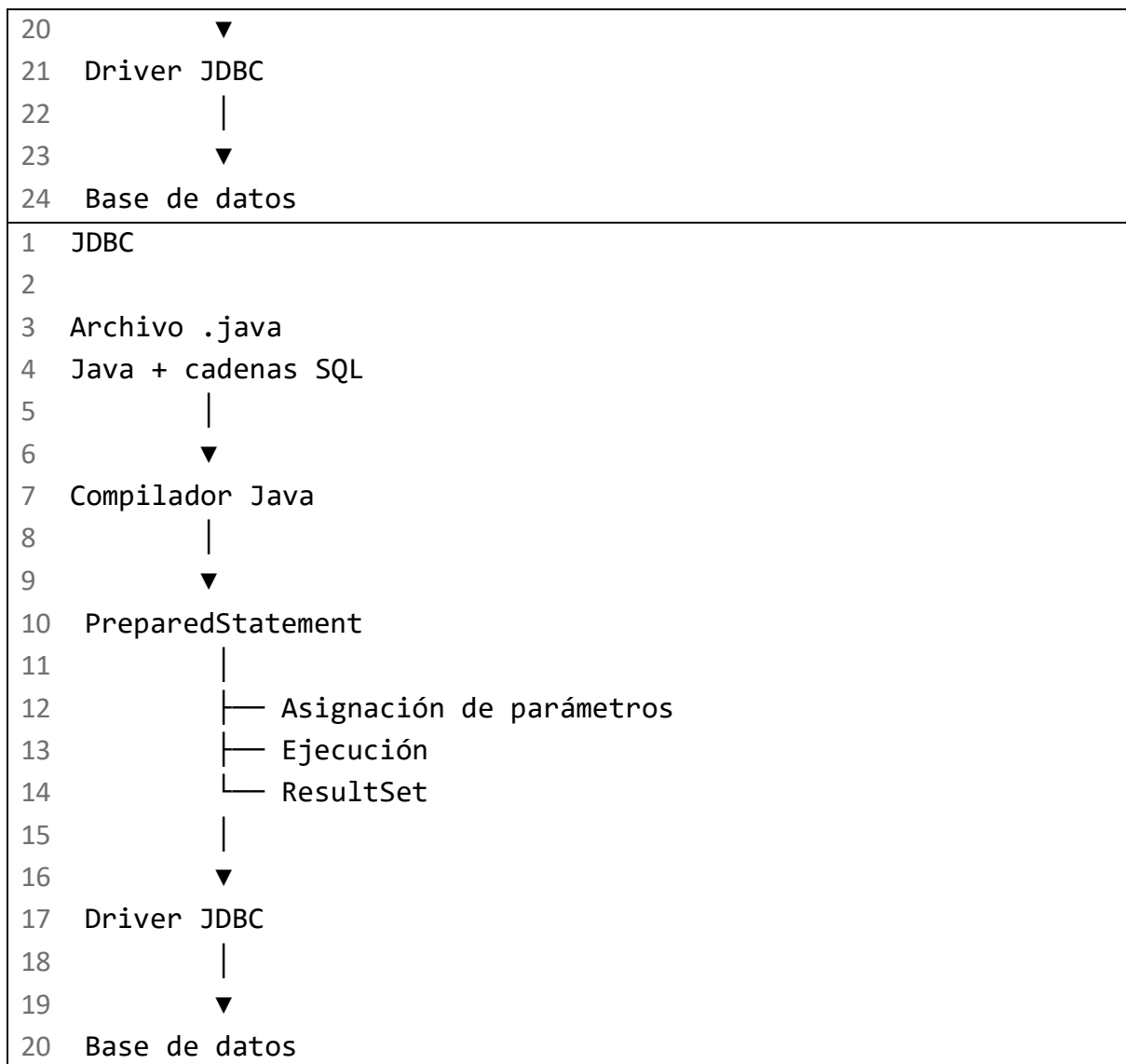
1. Comparativa entre SQLJ y JDBC

SQLJ y JDBC permiten ejecutar SQL desde Java, pero utilizan modelos de programación diferentes.

SQLJ incorpora SQL estático directamente en el código Java, lo procesa mediante un traductor y puede comprobar anticipadamente la sintaxis SQL, la existencia de objetos del esquema y la compatibilidad entre tipos Java y SQL, según la configuración del traductor. JDBC representa normalmente las sentencias mediante cadenas y las prepara o ejecuta durante el funcionamiento de la aplicación. [\[docs.oracle.com\]](https://docs.oracle.com), [\[download.oracle.com\]](https://download.oracle.com)

1.1. Visión general





El runtime SQLJ suele apoyarse en un controlador JDBC para comunicarse con la base de datos, aunque SQLJ añade una capa de traducción, tipado e integración diferente. Por tanto, SQLJ no debe presentarse como un sustituto interno de JDBC ni como una sintaxis alternativa equivalente. [\[docs.oracle.com\]](https://docs.oracle.com/javase/8/docs/technotes/guides/jsp/jsp2_2/sqlj.html), [\[docs.cloud...oracle.com\]](https://docs.oracle.com/javase/8/docs/technotes/guides/jsp/jsp2_2/sqlj.html)

1.2. Tabla comparativa

Criterio	SQLJ	JDBC
Tipo de SQL	Principalmente SQL estático	SQL estático o dinámico
Representación del SQL	Cláusulas #sql	Cadenas de texto
Procesamiento previo	Traductor SQLJ	No requiere traductor SQLJ
Validación anticipada	Posible durante la traducción	Normalmente al preparar o ejecutar

Variables de entrada	Variables host con :	Marcadores ? y métodos setXxx()
Valores de salida	SELECT INTO o iteradores	Métodos getXxx() de ResultSet
Varias filas	Iteradores SQLJ	ResultSet
SQL dinámico	Limitado	Adecuado
Número variable de filtros	Poco natural	Fácil de construir de forma controlada
Cantidad de código	Generalmente menor	Generalmente mayor
Control de ejecución	ExecutionContext	API detallada de Statement y PreparedStatement
Filas afectadas	ExecutionContext.getUpdateCount()	executeUpdate()
Metadatos dinámicos	No es su caso principal	DatabaseMetaData, ResultSetMetaData y ParameterMetaData
Transacciones	Contexto SQLJ y conexión asociada	Connection
Gestión de errores	SQLException	SQLException
Portabilidad	Depende del traductor y del dialecto	Depende principalmente del driver y del dialecto
Herramientas necesarias	Traductor, runtime y posiblemente perfiles	Driver JDBC
Encaje actual	Sistemas SQLJ existentes y entornos específicos	Opción general para acceso SQL directo
Curva de aprendizaje	Java, SQLJ, SQL y herramientas del fabricante	Java, JDBC y SQL
Mantenimiento	Conciso si el equipo conoce SQLJ	Más explícito y ampliamente conocido
Caso recomendado	SQL estable y plataforma SQLJ consolidada	Aplicaciones nuevas, SQL dinámico o portabilidad operativa

Oracle resume las principales ventajas de SQLJ como menor cantidad de código, comprobación sintáctica y semántica anticipada, tipado fuerte y asignación simplificada de parámetros. La misma comparación reconoce que JDBC ofrece control más detallado y verdadera capacidad de SQL dinámico. download.oracle.com

2. Misma consulta con SQLJ y JDBC

Se implementará la siguiente operación:

Recuperar todos los pedidos de un cliente cuyo importe sea igual o superior a un mínimo, ordenados por fecha e identificador.

2.1. SQL convencional

```
1  SELECT id_pedido,
2         fecha,
3         importe,
4         estado
5  FROM pedido
6  WHERE id_cliente = ?
7        AND importe >= ?
8  ORDER BY fecha, id_pedido;
```

2.2. Modelo de resultado

```
1  import java.math.BigDecimal;
2  import java.sql.Date;
3
4  public final class PedidoResumen {
5
6      private final int idPedido;
7      private final Date fecha;
8      private final BigDecimal importe;
9      private final String estado;
10
11     public PedidoResumen(
12         int idPedido,
13         Date fecha,
14         BigDecimal importe,
15         String estado) {
16
17         this.idPedido = idPedido;
18         this.fecha = fecha;
19         this.importe = importe;
20         this.estado = estado;
21     }
22
23     public int getIdPedido() {
24         return idPedido;
25     }
26 }
```

```

27     public Date getFecha() {
28         return fecha;
29     }
30
31     public BigDecimal getImporte() {
32         return importe;
33     }
34
35     public String getEstado() {
36         return estado;
37     }
38 }

```

2.3. Implementación SQLJ

Declaración del iterador

```

1  import java.math.BigDecimal;
2  import java.sql.Date;
3
4  #sql iterator PedidoResumenIterator(
5      int idPedido,
6      Date fecha,
7      BigDecimal importe,
8      String estado
9  );

```

Método SQLJ

```

1  import java.math.BigDecimal;
2  import java.sql.SQLException;
3  import java.util.ArrayList;
4  import java.util.List;
5
6  import sqlj.runtime.ref.DefaultContext;
7
8  public List<PedidoResumen> buscarPedidosSQLJ(
9      DefaultContext contexto,
10     int idCliente,
11     BigDecimal importeMinimo)
12     throws SQLException {
13

```

```
14     if (idCliente <= 0) {
15         throw new IllegalArgumentException(
16             "El identificador del cliente debe ser positivo"
17         );
18     }
19
20     if (importeMinimo == null ||
21         importeMinimo.signum() < 0) {
22
23         throw new IllegalArgumentException(
24             "El importe mínimo debe ser mayor o igual que
25             cero"
26         );
27     }
28
29     List<PedidoResumen> resultado =
30         new ArrayList<PedidoResumen>();
31
32     PedidoResumenIterator iterador = null;
33     SQLException errorPrincipal = null;
34
35     try {
36         #sql [contexto] iterador = {
37             SELECT id_pedido AS idPedido,
38                 fecha AS fecha,
39                 importe AS importe,
40                 estado AS estado
41             FROM pedido
42             WHERE id_cliente = :idCliente
43                 AND importe >= :importeMinimo
44             ORDER BY fecha, id_pedido
45         };
46
47         while (iterador.next()) {
48             resultado.add(
49                 new PedidoResumen(
50                     iterador.idPedido(),
51                     iterador.fecha(),
52                     iterador.importe(),
53                     iterador.estado()
```

```

53         )
54     );
55 }
56
57     return resultado;
58
59 } catch (SQLException exception) {
60     errorPrincipal = exception;
61     throw exception;
62
63 } finally {
64     if (iterador != null) {
65         try {
66             iterador.close();
67         } catch (SQLException closeException) {
68             if (errorPrincipal != null) {
69                 errorPrincipal.addSuppressed(
70                     closeException
71                 );
72             } else {
73                 throw closeException;
74             }
75         }
76     }
77 }
78 }

```

Flujo SQLJ

```

1  idCliente + importeMinimo
2      |
3      ▼
4  Variables host
5      |
6      ▼
7  Consulta SQL estática
8      |
9      ▼
10 PedidoResumenIterator
11      |
12      └─ idPedido()

```

13	— fecha()
14	— importe()
15	— estado()

2.4. Implementación JDBC

PreparedStatement representa una sentencia SQL preparada. Los parámetros de entrada se asignan mediante métodos compatibles con el tipo SQL, como `setInt()` o `setBigDecimal()`.

[\[docs.oracle.com\]](https://docs.oracle.com)

```
1  import java.math.BigDecimal;
2  import java.sql.Connection;
3  import java.sql.PreparedStatement;
4  import java.sql.ResultSet;
5  import java.sql.SQLException;
6  import java.util.ArrayList;
7  import java.util.List;
8
9  public List<PedidoResumen> buscarPedidosJDBC(
10      Connection connection,
11      int idCliente,
12      BigDecimal importeMinimo)
13      throws SQLException {
14
15      if (connection == null) {
16          throw new IllegalArgumentException(
17              "La conexión no puede ser null"
18          );
19      }
20
21      if (idCliente <= 0) {
22          throw new IllegalArgumentException(
23              "El identificador del cliente debe ser positivo"
24          );
25      }
26
27      if (importeMinimo == null ||
28          importeMinimo.signum() < 0) {
29
30          throw new IllegalArgumentException(
```



```

31         "El importe mínimo debe ser mayor o igual que
cero"
32     );
33 }
34
35 String sql =
36     "SELECT id_pedido, " +
37     "    fecha, " +
38     "    importe, " +
39     "    estado " +
40     "FROM pedido " +
41     "WHERE id_cliente = ? " +
42     " AND importe >= ? " +
43     "ORDER BY fecha, id_pedido";
44
45 List<PedidoResumen> resultado =
46     new ArrayList<PedidoResumen>();
47
48 try (PreparedStatement statement =
49     connection.prepareStatement(sql)) {
50
51     statement.setInt(1, idCliente);
52     statement.setBigDecimal(2, importeMinimo);
53
54     try (ResultSet resultSet =
55         statement.executeQuery()) {
56
57         while (resultSet.next()) {
58             resultado.add(
59                 new PedidoResumen(
60                     resultSet.getInt("id_pedido"),
61                     resultSet.getDate("fecha"),
62                     resultSet.getBigDecimal("importe"),
63                     resultSet.getString("estado")
64                 )
65             );
66         }
67     }
68 }
69

```

```
70     return resultado;
71 }
```

ResultSet mantiene un cursor inicialmente situado antes de la primera fila. next() avanza el cursor y devuelve false al terminar. Los valores pueden recuperarse por posición o por nombre de columna. [\[docs.oracle.com\]](https://docs.oracle.com)

2.5. Comparación directa del código

SQLJ concentra la operación

```
1  #sql [contexto] iterador = {
2      SELECT id_pedido AS idPedido,
3             fecha AS fecha,
4             importe AS importe,
5             estado AS estado
6      FROM pedido
7      WHERE id_cliente = :idCliente
8             AND importe >= :importeMinimo
9  };
```

JDBC separa varias responsabilidades

```
1  PreparedStatement statement =
2      connection.prepareStatement(sql);
3
4  statement.setInt(1, idCliente);
5  statement.setBigDecimal(2, importeMinimo);
6
7  ResultSet resultSet =
8      statement.executeQuery();
```

Después recupera cada valor:

```
1  int idPedido =
2      resultSet.getInt("id_pedido");
3
4  BigDecimal importe =
5      resultSet.getBigDecimal("importe");
```

SQLJ suele resultar más breve para consultas SQL estáticas. JDBC hace más explícitos la preparación, la asignación de parámetros, la ejecución y el procesamiento del resultado. [\[download.oracle.com\]](https://download.oracle.com), [\[docs.oracle.com\]](https://docs.oracle.com), [\[docs.oracle.com\]](https://docs.oracle.com)

3. Consultas dinámicas

La diferencia entre ambas tecnologías se vuelve especialmente visible cuando cambia la estructura de la consulta.

Supongamos una búsqueda con filtros opcionales:

- Ciudad.
- Estado del pedido.
- Importe mínimo.
- Orden elegido por el usuario.

3.1. Limitación con SQLJ estático

Una variable host representa un valor:

```
1 WHERE ciudad = :ciudad
```

No puede representar directamente:

- Una columna.
- Una tabla.
- Una cláusula WHERE.
- Un operador.
- ASC o DESC.
- Una cantidad variable de marcadores.

Esta construcción no es válida:

```
1 String orden = "importe DESC";
2
3 #sql [contexto] iterador = {
4     SELECT id_pedido AS idPedido,
5           fecha AS fecha,
6           importe AS importe,
7           estado AS estado
8     FROM pedido
9     ORDER BY :orden
10 };
```

:orden se interpreta como un valor, no como sintaxis SQL.

3.2. Alternativa SQLJ con ramas estáticas

```
1 if (ordenarPorImporte) {
2     #sql [contexto] iterador = {
```

```

3      SELECT id_pedido AS idPedido,
4             fecha AS fecha,
5             importe AS importe,
6             estado AS estado
7      FROM pedido
8      WHERE id_cliente = :idCliente
9      ORDER BY importe DESC, id_pedido
10     };
11 } else {
12     #sql [contexto] iterador = {
13         SELECT id_pedido AS idPedido,
14                fecha AS fecha,
15                importe AS importe,
16                estado AS estado
17         FROM pedido
18         WHERE id_cliente = :idCliente
19         ORDER BY fecha DESC, id_pedido
20     };
21 }

```

Esta solución es adecuada cuando existe un número reducido de variantes.

3.3. Alternativa JDBC dinámica y controlada

```

1  public List<PedidoResumen> buscarPedidosDinamicos(
2      Connection connection,
3      int idCliente,
4      String estado,
5      BigDecimal importeMinimo,
6      String criterioOrden)
7      throws SQLException {
8
9      StringBuilder sql = new StringBuilder();
10
11     sql.append(
12         "SELECT id_pedido, fecha, importe, estado " +
13         "FROM pedido " +
14         "WHERE id_cliente = ? "
15     );
16
17     List<Object> parametros = new ArrayList<Object>();

```

```
18     parametros.add(idCliente);
19
20     if (estado != null) {
21         sql.append("AND estado = ? ");
22         parametros.add(estado);
23     }
24
25     if (importeMinimo != null) {
26         sql.append("AND importe >= ? ");
27         parametros.add(importeMinimo);
28     }
29
30     if ("IMPORTE".equals(criterioOrden)) {
31         sql.append("ORDER BY importe DESC, id_pedido");
32     } else if ("FECHA".equals(criterioOrden)) {
33         sql.append("ORDER BY fecha DESC, id_pedido");
34     } else {
35         sql.append("ORDER BY id_pedido");
36     }
37
38     List<PedidoResumen> resultado =
39         new ArrayList<PedidoResumen>();
40
41     try (PreparedStatement statement =
42         connection.prepareStatement(sql.toString())) {
43
44         for (int i = 0; i < parametros.size(); i++) {
45             statement.setObject(i + 1, parametros.get(i));
46         }
47
48         try (ResultSet resultSet =
49             statement.executeQuery()) {
50
51             while (resultSet.next()) {
52                 resultado.add(
53                     new PedidoResumen(
54                         resultSet.getInt("id_pedido"),
55                         resultSet.getDate("fecha"),
56                         resultSet.getBigDecimal("importe"),
57                         resultSet.getString("estado")
```

```
58         )
59     );
60 }
61 }
62 }
63
64     return resultado;
65 }
```

Los valores siguen enviándose mediante parámetros. Solo las partes estructurales controladas por la aplicación se incorporan a la cadena SQL.

No debe concatenarse directamente un criterio recibido del usuario:

```
1  sql.append(" ORDER BY " + ordenUsuario);
```

4. Recomendación de elección

4.1. Utilizar SQLJ cuando

- La aplicación ya está construida con SQLJ.
- El equipo domina sus herramientas.
- Las consultas tienen una estructura estable.
- La plataforma y la base de datos tienen soporte SQLJ conocido.
- Se aprovecha la validación anticipada.
- Existen procesos establecidos de traducción, personalización o bind.
- Migrar no proporciona un beneficio suficiente.
- Se desea mantener una aplicación empresarial existente con cambios controlados.

Oracle continúa documentando SQLJ como una tecnología formada por un traductor y un runtime, centrada en SQL estático y complementable con JDBC para operaciones dinámicas.

[\[docs.oracle.com\]](https://docs.oracle.com), [\[docs.oracle.com\]](https://docs.oracle.com)

4.2. Utilizar JDBC cuando

- La aplicación es nueva y no existe una razón concreta para incorporar SQLJ.
- Se necesita SQL dinámico.
- Los filtros o proyecciones cambian durante la ejecución.
- Se necesita acceso directo a metadatos.
- El equipo domina JDBC.
- Se quiere reducir la dependencia de un traductor SQLJ.
- Se necesita un control detallado de sentencias, resultados y capacidades del driver.
- Se necesita una integración sencilla con herramientas Java actuales.

4.3. Utilizar ambos cuando

Una aplicación SQLJ puede incluir JDBC para las partes dinámicas:

```
1 Consultas estables
2   |
3   └─ SQLJ
4
5 Búsquedas dinámicas
6   |
7   └─ JDBC
```

Oracle señala expresamente que las operaciones dinámicas pueden realizarse con JDBC dentro de una aplicación SQLJ. [\[docs.oracle.com\]](https://docs.oracle.com)

4.4. Recomendación práctica

Para un sistema SQLJ existente:

```
1 No migrar por defecto
2   |
3   ▼
4 Evaluar cada módulo
5   |
6   ├── SQL estable y mantenible
7   │   └─ Conservar SQLJ
8   |
9   ├── SQL dinámico complejo
10  │   └─ Considerar JDBC
11  |
12  └─ Módulo nuevo
13      └─ Elegir según arquitectura actual
```

Para una aplicación nueva, JDBC o una herramienta moderna de acceso a datos suele ofrecer un ecosistema más sencillo. SQLJ sigue siendo razonable cuando existe una necesidad concreta de SQL estático, soporte del fabricante y experiencia operativa.

5. Buenas prácticas

5.1. Usar parámetros

Correcto:

```
1 #sql [contexto] {
```

```
2     SELECT nombre
3     INTO :nombre
4     FROM cliente
5     WHERE id_cliente = :idCliente
6 };
```

En JDBC:

```
1 String sql =
2     "SELECT nombre " +
3     "FROM cliente " +
4     "WHERE id_cliente = ?";
5
6 try (PreparedStatement statement =
7     connection.prepareStatement(sql)) {
8
9     statement.setInt(1, idCliente);
10 }
```

Incorrecto:

```
1 String sql =
2     "SELECT nombre FROM cliente " +
3     "WHERE id_cliente = " + idCliente;
```

Los parámetros separan los datos de la estructura SQL. En JDBC, los métodos setXxx() deben emplear tipos compatibles con los parámetros SQL. docs.oracle.com

5.2. No utilizar variables host como sintaxis SQL

Correcto:

```
1 WHERE ciudad = :ciudad
```

Incorrecto:

```
1 FROM :nombreTabla
```

Una variable host representa un dato, no un identificador SQL.

5.3. Utilizar tipos compatibles

Correspondencias habituales:

1 SQL INTEGER	→ int o Integer
2 SQL BIGINT	→ long o Long
3 SQL VARCHAR	→ String

4	SQL DECIMAL	→ BigDecimal
5	SQL DATE	→ java.sql.Date
6	SQL TIMESTAMP	→ java.sql.Timestamp

La correspondencia exacta debe comprobarse con el controlador y la implementación SQLJ.

Para importes:

1	BigDecimal importe;
---	---------------------

Evitar:

1	double importe;
---	-----------------

5.4. Tratar correctamente los valores NULL

1	SQL NULL
2	
3	▼
4	Tipo Java anulable

Ejemplos:

1	Integer cantidad;
2	Long identificador;
3	BigDecimal importe;
4	String ciudad;

Un primitivo no puede contener null:

1	int cantidad;
---	---------------

En JDBC, getInt() devuelve 0 tanto para un cero real como para un NULL. Para distinguirlos se puede usar wasNull():

1	int cantidad = resultSet.getInt("cantidad");
2	
3	Integer cantidadReal =
4	resultSet.wasNull() ? null : cantidad;

Cuando sea posible, puede usarse:

1	Integer cantidad =
2	resultSet.getObject(
3	"cantidad",
4	Integer.class
5	);

La disponibilidad y el comportamiento concreto dependen del driver.

5.5. Evitar SELECT *

Evitar:

```
1 SELECT *  
2 FROM pedido;
```

Preferir:

```
1 SELECT id_pedido,  
2        fecha,  
3        importe,  
4        estado  
5 FROM pedido;
```

Beneficios:

- Contrato de columnas explícito.
 - Menor volumen transferido.
 - Menor dependencia de cambios del esquema.
 - Iteradores más fáciles de mantener.
 - Menor riesgo de ambigüedad en los JOIN.
-

5.6. Seleccionar solo las columnas necesarias

Si la aplicación necesita únicamente el nombre:

```
1 SELECT nombre  
2 FROM cliente  
3 WHERE id_cliente = ?;
```

No:

```
1 SELECT id_cliente,  
2        nombre,  
3        email,  
4        ciudad  
5 FROM cliente  
6 WHERE id_cliente = ?;
```

5.7. Cerrar iteradores

```
1 ClienteIterator iterador = null;  
2 SQLException errorPrincipal = null;  
3
```

```

4  try {
5      #sql [contexto] iterador = {
6          SELECT id_cliente AS idCliente,
7              nombre AS nombre,
8              email AS email,
9              ciudad AS ciudad
10         FROM cliente
11     };
12
13     while (iterador.next()) {
14         // Procesamiento.
15     }
16
17 } catch (SQLException exception) {
18     errorPrincipal = exception;
19     throw exception;
20
21 } finally {
22     if (iterador != null) {
23         try {
24             iterador.close();
25         } catch (SQLException closeException) {
26             if (errorPrincipal != null) {
27                 errorPrincipal.addSuppressed(
28                     closeException
29                 );
30             } else {
31                 throw closeException;
32             }
33         }
34     }
35 }

```

5.8. Cerrar recursos JDBC con try-with-resources

PreparedStatement y ResultSet implementan AutoCloseable, por lo que pueden gestionarse mediante try-with-resources en Java 8. [docs.oracle.com](https://docs.oracle.com/javase/8/docs/api/java/sql/PreparedStatement.html), [docs.oracle.com](https://docs.oracle.com/javase/8/docs/api/java/sql/ResultSet.html)

```

1  try (PreparedStatement statement =
2      connection.prepareStatement(sql)) {
3

```

```

4      try (ResultSet resultSet =
5          statement.executeQuery()) {
6
7          while (resultSet.next()) {
8              // Procesamiento.
9          }
10     }
11 }

```

5.9. Definir la propiedad de la conexión

```

1  Componente que crea la conexión
2      |
3      └─ Responsable de cerrarla
4
5  Método que recibe la conexión
6      |
7      └─ No debe cerrarla salvo contrato explícito

```

En un repositorio:

```

1  public RepositorioSQLJ(
2      Connection connection,
3      DefaultContext contexto) {
4
5      this.connection = connection;
6      this.contexto = contexto;
7  }

```

El repositorio puede cerrar sus iteradores, pero no debería cerrar automáticamente una conexión que pertenece al llamador.

5.10. Delimitar correctamente las transacciones

Una transacción debe representar una unidad de negocio:

```

1  Crear pedido
2  └─ Comprobar cliente
3  └─ Comprobar productos
4  └─ Insertar cabecera
5  └─ Insertar líneas
6  └─ Calcular total
7  └─ Actualizar importe

```

Resultado:

- | | |
|---|------------------------|
| 1 | Todo correcto → COMMIT |
| 2 | Algún error → ROLLBACK |

No deben mezclarse en una única transacción operaciones independientes que prolonguen innecesariamente:

- Bloqueos.
 - Uso de conexiones.
 - Cursores abiertos.
 - Tiempo de espera de otros procesos.
-

5.11. No confirmar parcialmente una operación

Incorrecto:

1	INSERT PEDIDO
2	
3	▼
4	COMMIT
5	
6	▼
7	INSERT LINEA 1
8	
9	▼
10	Error

Resultado:

1	Pedido incompleto
---	-------------------

Correcto:

1	INSERT PEDIDO
2	
3	▼
4	INSERT LINEA 1
5	
6	▼
7	INSERT LINEA 2
8	
9	▼
10	COMMIT

5.12. Restaurar el estado de conexiones reutilizables

Antes de devolver una conexión a un pool:

- No debe haber transacciones pendientes.
 - Debe restaurarse el autocommit.
 - Deben cerrarse los iteradores.
 - Deben cerrarse los resultados JDBC.
 - Deben restablecerse propiedades modificadas cuando corresponda.
 - Debe invalidarse la conexión si su estado es incierto.
-

5.13. No ocultar excepciones

Patrón recomendado:

```
1  try {  
2      connection.rollback();  
3  } catch (SQLException rollbackException) {  
4      exception.addSuppressed(  
5          rollbackException  
6      );  
7  }  
8  
9  throw exception;
```

Esto conserva el error principal y el error secundario.

5.14. Registrar errores sin exponer información sensible

Puede registrarse:

```
1  Identificador de correlación  
2  Nombre lógico de la operación  
3  SQLState  
4  Código del fabricante  
5  Duración  
6  Número de intento
```

No debería registrarse:

```
1  Contraseña  
2  URL con credenciales  
3  Tokens
```

- 4 Datos personales completos
- 5 Números de cuenta
- 6 Contenido íntegro de parámetros sensibles

SQLJ utiliza SQLException para la gestión de errores, permitiendo consultar al menos el código del fabricante y SQLState. [\[ibm.com\]](#), [\[ibm.com\]](#)

5.15. Usar índices en filtros y JOIN

Columnas candidatas en el modelo:

- 1 PEDIDO.id_cliente
- 2 LINEA_PEDIDO.id_pedido
- 3 LINEA_PEDIDO.id_producto
- 4 PEDIDO.estado
- 5 PEDIDO.fecha
- 6 PRODUCTO.categoria

No debe crearse un índice automáticamente para cada columna. Deben evaluarse:

- Volumen.
 - Selectividad.
 - Frecuencia de lectura.
 - Frecuencia de escritura.
 - Planes de ejecución.
 - Índices ya existentes.
 - Orden de las columnas en índices compuestos.
-

5.16. Diferenciar filtros en ON y WHERE

Filtro en ON

```
1 SELECT c.nombre,  
2        p.id_pedido  
3 FROM cliente c  
4 LEFT JOIN pedido p  
5     ON p.id_cliente = c.id_cliente  
6     AND p.estado = 'PENDIENTE';
```

Conserva todos los clientes.

Filtro en WHERE

```
1 SELECT c.nombre,
```

```

2      p.id_pedido
3  FROM cliente c
4  LEFT JOIN pedido p
5      ON p.id_cliente = c.id_cliente
6  WHERE p.estado = 'PENDIENTE';

```

Elimina los clientes sin pedidos pendientes.

Representación:

```

1  Condición en ON
2      |
3      └─ Controla qué filas se relacionan
4
5  Condición en WHERE
6      |
7      └─ Filtra el resultado ya combinado

```

5.17. Usar correctamente UNION y UNION ALL

```

1  ¿Se deben eliminar duplicados?
2      |
3      └─┬── Sí
4          │   UNION
5          └─ No
6              │   UNION ALL
7              └─

```

UNION ALL suele ser preferible cuando:

- Los conjuntos no se solapan.
- Los duplicados tienen significado.
- No se necesita deduplicación.

5.18. Evitar resultados excesivos

Una consulta debe limitarse por:

- Cliente.
- Pedido.
- Intervalo temporal.
- Categoría.
- Estado.
- Paginación.

Consulta potencialmente problemática:

```
1  SELECT id_pedido,  
2         fecha,  
3         importe,  
4         estado  
5  FROM pedido  
6  ORDER BY fecha;
```

Si la tabla contiene millones de filas, debería incluirse un filtro o mecanismo de paginación.

La sintaxis de paginación depende de la base de datos y de su versión.

5.19. Prevenir UPDATE y DELETE masivos

Antes de una operación sensible:

1. Comprobar el filtro.
2. Usar ExecutionContext.
3. Examinar las filas afectadas.
4. Ejecutar dentro de una transacción.
5. Rechazar recuentos inesperados.

```
1  ExecutionContext ejecucion =  
2      new ExecutionContext();  
3  
4  #sql [contexto, ejecucion] {  
5      DELETE FROM pedido  
6      WHERE id_pedido = :idPedido  
7  };  
8  
9  int filas = ejecucion.getUpdateCount();  
10  
11  if (filas != 1) {  
12      throw new SQLException(  
13          "Se esperaba eliminar un pedido, " +  
14          "pero se eliminaron " + filas  
15      );  
16  }
```

6. Ejercicio final integral

6.1. Enunciado

Desarrollar una operación SQLJ que registre un pedido completo.

La operación debe:

1. Recibir cliente, pedido y líneas.
 2. Comprobar que el cliente existe.
 3. Comprobar que cada producto existe.
 4. Comprobar que cada producto tiene un precio válido.
 5. Impedir productos duplicados dentro del pedido.
 6. Calcular el importe de cada línea.
 7. Calcular el importe total.
 8. Insertar la cabecera del pedido.
 9. Insertar todas las líneas.
 10. Actualizar el importe total.
 11. Confirmar mediante COMMIT.
 12. Ejecutar ROLLBACK si falla cualquier paso.
 13. Consultar posteriormente el pedido junto con cliente y productos.
-

6.2. Modelo utilizado

```
1  CLIENTE
2  |— PK id_cliente
3  |— nombre
4  |— email
5  |— ciudad
6
7  PEDIDO
8  |— PK id_pedido
9  |— FK id_cliente
10 |— fecha
11 |— importe
12 |— estado
13
14 PRODUCTO
15 |— PK id_producto
16 |— nombre
17 |— precio
```

```

18  └─ categoria
19
20  LINEA_PEDIDO
21  └─ PK/FK id_pedido
22  └─ PK/FK id_producto
23  └─ cantidad

```

6.3. Restricciones recomendadas

```

1  ALTER TABLE pedido
2  ADD CONSTRAINT fk_pedido_cliente
3  FOREIGN KEY (id_cliente)
4  REFERENCES cliente (id_cliente);

```

```

1  ALTER TABLE linea_pedido
2  ADD CONSTRAINT fk_linea_pedido
3  FOREIGN KEY (id_pedido)
4  REFERENCES pedido (id_pedido);

```

```

1  ALTER TABLE linea_pedido
2  ADD CONSTRAINT fk_linea_producto
3  FOREIGN KEY (id_producto)
4  REFERENCES producto (id_producto);

```

```

1  ALTER TABLE linea_pedido
2  ADD CONSTRAINT ck_linea_cantidad
3  CHECK (cantidad > 0);

```

La sintaxis concreta para modificar restricciones y asignar nombres puede variar entre bases de datos.

6.4. Flujo de la solución

```

1  Datos de entrada
2      |
3      ▼
4  Validaciones Java
5      |
6      ▼
7  Desactivar autocommit
8      |
9      ▼
10 Comprobar cliente

```


7. Solución razonada paso a paso

Paso 1. Validar los datos en Java

Las validaciones que no requieren consultar la base de datos deben ejecutarse antes de iniciar la transacción:

```
1 idPedido > 0
2 idCliente > 0
3 fecha no nula
4 estado no vacío
5 lista de líneas no vacía
6 cantidad > 0
7 producto no duplicado
```

Esto evita abrir una transacción para datos evidentemente inválidos.

Paso 2. Comprobar duplicados

```
1 private void validarProductosNoDuplicados(
2     List<LineaPedido> lineas) {
3
4     Set<Integer> productos =
5         new HashSet<Integer>();
6
7     for (LineaPedido linea : lineas) {
8         boolean añadido = productos.add(
9             linea.getIdProducto()
10        );
11
12        if (!añadido) {
13            throw new IllegalArgumentException(
14                "Producto repetido: " +
15                linea.getIdProducto()
16            );
17        }
18    }
19 }
```

Paso 3. Iniciar la transacción

```
1  boolean autoCommitOriginal =
2      connection.getAutoCommit();
3
4  connection.setAutoCommit(false);
```

Paso 4. Comprobar el cliente

```
1  private void comprobarCliente(int idCliente)
2      throws SQLException {
3
4      int numeroClientes = 0;
5
6      #sql [contexto] {
7          SELECT COUNT(*)
8          INTO :numeroClientes
9          FROM cliente
10         WHERE id_cliente = :idCliente
11     };
12
13     if (numeroClientes != 1) {
14         throw new SQLException(
15             "El cliente no existe: " + idCliente
16         );
17     }
18 }
```

Paso 5. Insertar la cabecera

Inicialmente el importe es cero:

```
1  BigDecimal importeInicial =
2      BigDecimal.ZERO;
```

Después se actualiza al terminar las líneas.

Paso 6. Comprobar productos y calcular total

```
1  precio × cantidad = subtotal
```

Con BigDecimal:

```
1 BigDecimal subtotal =
2     precio.multiply(
3         BigDecimal.valueOf(cantidad)
4     );
```

Paso 7. Insertar las líneas

Cada línea debe afectar exactamente una fila.

Paso 8. Actualizar el total

```
1 UPDATE pedido
2 SET importe = :importeTotal
3 WHERE id_pedido = :idPedido
```

Paso 9. Confirmar o deshacer

```
1 Sin errores → commit()
2 Con errores → rollback()
```

8. Código completo del ejercicio

8.1. Clases de entrada y salida

```
1 package ejemplo.sqlj.ejercicio;
2
3 import java.math.BigDecimal;
4 import java.sql.Date;
5 import java.util.ArrayList;
6 import java.util.Collections;
7 import java.util.List;
8
9 public final class DatosPedido {
10
11     private final int idPedido;
12     private final int idCliente;
13     private final Date fecha;
14     private final String estado;
15     private final List<Linea> lineas;
```

```
16
17     public DatosPedido(
18         int idPedido,
19         int idCliente,
20         Date fecha,
21         String estado,
22         List<Linea> lineas) {
23
24         this.idPedido = idPedido;
25         this.idCliente = idCliente;
26         this.fecha = fecha;
27         this.estado = estado;
28         this.lineas = Collections.unmodifiableList(
29             new ArrayList<Linea>(lineas)
30         );
31     }
32
33     public int getIdPedido() {
34         return idPedido;
35     }
36
37     public int getIdCliente() {
38         return idCliente;
39     }
40
41     public Date getFecha() {
42         return fecha;
43     }
44
45     public String getEstado() {
46         return estado;
47     }
48
49     public List<Linea> getLineas() {
50         return lineas;
51     }
52
53     public static final class Linea {
54
55         private final int idProducto;
```



```
56         private final int cantidad;
57
58         public Linea(
59             int idProducto,
60             int cantidad) {
61
62             this.idProducto = idProducto;
63             this.cantidad = cantidad;
64         }
65
66         public int getIdProducto() {
67             return idProducto;
68         }
69
70         public int getCantidad() {
71             return cantidad;
72         }
73     }
74
75     public static final class Detalle {
76
77         private final int idPedido;
78         private final String cliente;
79         private final Date fecha;
80         private final String estado;
81         private final BigDecimal importeTotal;
82         private final int idProducto;
83         private final String producto;
84         private final BigDecimal precio;
85         private final int cantidad;
86         private final BigDecimal subtotal;
87
88         public Detalle(
89             int idPedido,
90             String cliente,
91             Date fecha,
92             String estado,
93             BigDecimal importeTotal,
94             int idProducto,
95             String producto,
```

```
96         BigDecimal precio,
97         int cantidad,
98         BigDecimal subtotal) {
99
100         this.idPedido = idPedido;
101         this.cliente = cliente;
102         this.fecha = fecha;
103         this.estado = estado;
104         this.importeTotal = importeTotal;
105         this.idProducto = idProducto;
106         this.producto = producto;
107         this.precio = precio;
108         this.cantidad = cantidad;
109         this.subtotal = subtotal;
110     }
111
112     public int getIdPedido() {
113         return idPedido;
114     }
115
116     public String getCliente() {
117         return cliente;
118     }
119
120     public Date getFecha() {
121         return fecha;
122     }
123
124     public String getEstado() {
125         return estado;
126     }
127
128     public BigDecimal getImporteTotal() {
129         return importeTotal;
130     }
131
132     public int getIdProducto() {
133         return idProducto;
134     }
135
```

```
136         public String getProducto() {
137             return producto;
138         }
139
140         public BigDecimal getPrecio() {
141             return precio;
142         }
143
144         public int getCantidad() {
145             return cantidad;
146         }
147
148         public BigDecimal getSubtotal() {
149             return subtotal;
150         }
151     }
152 }
```

8.2. Iterador SQLJ

```
1  package ejemplo.sqlj.ejercicio;
2
3  import java.math.BigDecimal;
4  import java.sql.Date;
5
6  #sql public iterator DetallePedidoIterator(
7      int idPedido,
8      String cliente,
9      Date fecha,
10     String estado,
11     BigDecimal importeTotal,
12     int idProducto,
13     String producto,
14     BigDecimal precio,
15     int cantidad,
16     BigDecimal subtotal
17 );
```

8.3. Servicio SQLJ

```
1  package ejemplo.sqlj.ejercicio;
2
3  import java.math.BigDecimal;
4  import java.sql.Connection;
5  import java.sql.SQLException;
6  import java.util.ArrayList;
7  import java.util.HashSet;
8  import java.util.List;
9  import java.util.Set;
10
11  import sqlj.runtime.ExecutionContext;
12  import sqlj.runtime.ref.DefaultContext;
13
14  import ejemplo.sqlj.ejercicio.DatosPedido.Detalle;
15  import ejemplo.sqlj.ejercicio.DatosPedido.Linea;
16
17  public final class ServicioPedidosSQLJ {
18
19      private final Connection connection;
20      private final DefaultContext contexto;
21
22      public ServicioPedidosSQLJ(
23          Connection connection,
24          DefaultContext contexto) {
25
26          if (connection == null) {
27              throw new IllegalArgumentException(
28                  "La conexión es obligatoria"
29              );
30          }
31
32          if (contexto == null) {
33              throw new IllegalArgumentException(
34                  "El contexto SQLJ es obligatorio"
35              );
36          }
37
38          this.connection = connection;
39          this.contexto = contexto;
```

```
40     }
41
42     public BigDecimal crearPedido(
43         DatosPedido datos)
44         throws SQLException {
45
46         validar(datos);
47
48         boolean autoCommitOriginal =
49             connection.getAutoCommit();
50
51         SQLException errorPrincipal = null;
52
53         try {
54             connection.setAutoCommit(false);
55
56             comprobarCliente(
57                 datos.getIdCliente()
58             );
59
60             insertarCabecera(datos);
61
62             BigDecimal importeTotal =
63                 insertarLineasYCalcularTotal(
64                     datos
65                 );
66
67             actualizarImporte(
68                 datos.getIdPedido(),
69                 importeTotal
70             );
71
72             connection.commit();
73
74             return importeTotal;
75
76         } catch (SQLException exception) {
77             errorPrincipal = exception;
78
79             rollbackConservandoError(exception);
```

```
80
81         throw exception;
82
83     } catch (RuntimeException exception) {
84         rollbackConservandoError(exception);
85
86         throw exception;
87
88     } finally {
89         restaurarAutoCommit(
90             autoCommitOriginal,
91             errorPrincipal
92         );
93     }
94 }
95
96 private void validar(DatosPedido datos) {
97
98     if (datos == null) {
99         throw new IllegalArgumentException(
100             "Los datos del pedido son obligatorios"
101         );
102     }
103
104     if (datos.getIdPedido() <= 0) {
105         throw new IllegalArgumentException(
106             "El identificador del pedido no es válido"
107         );
108     }
109
110     if (datos.getIdCliente() <= 0) {
111         throw new IllegalArgumentException(
112             "El identificador del cliente no es válido"
113         );
114     }
115
116     if (datos.getFecha() == null) {
117         throw new IllegalArgumentException(
118             "La fecha es obligatoria"
119         );
120     }
121 }
```

```
120     }
121
122     if (datos.getEstado() == null ||
123         datos.getEstado().trim().isEmpty()) {
124
125         throw new IllegalArgumentException(
126             "El estado es obligatorio"
127         );
128     }
129
130     if (datos.getLineas() == null ||
131         datos.getLineas().isEmpty()) {
132
133         throw new IllegalArgumentException(
134             "El pedido debe tener al menos una línea"
135         );
136     }
137
138     Set<Integer> productos =
139         new HashSet<Integer>();
140
141     for (Linea linea : datos.getLineas()) {
142         if (linea == null) {
143             throw new IllegalArgumentException(
144                 "Una línea no puede ser null"
145             );
146         }
147
148         if (linea.getIdProducto() <= 0) {
149             throw new IllegalArgumentException(
150                 "Identificador de producto no válido"
151             );
152         }
153
154         if (linea.getCantidad() <= 0) {
155             throw new IllegalArgumentException(
156                 "La cantidad debe ser mayor que cero"
157             );
158         }
159     }
```

```
160         if (!productos.add(  
161             linea.getIdProducto())) {  
162  
163             throw new IllegalArgumentException(  
164                 "Producto duplicado: " +  
165                 linea.getIdProducto()  
166             );  
167         }  
168     }  
169 }  
170  
171 private void comprobarCliente(int idCliente)  
172     throws SQLException {  
173  
174     int numeroClientes = 0;  
175  
176     #sql [contexto] {  
177         SELECT COUNT(*)  
178         INTO :numeroClientes  
179         FROM cliente  
180         WHERE id_cliente = :idCliente  
181     };  
182  
183     if (numeroClientes != 1) {  
184         throw new SQLException(  
185             "No existe el cliente " + idCliente  
186         );  
187     }  
188 }  
189  
190 private void insertarCabecera(  
191     DatosPedido datos)  
192     throws SQLException {  
193  
194     int idPedido = datos.getIdPedido();  
195     int idCliente = datos.getIdCliente();  
196     java.sql.Date fecha = datos.getFecha();  
197     String estado = datos.getEstado();  
198     BigDecimal importeInicial =  
199         BigDecimal.ZERO;
```



```
200
201     ExecutionContext ejecucion =
202         new ExecutionContext();
203
204     #sql [contexto, ejecucion] {
205         INSERT INTO pedido (
206             id_pedido,
207             id_cliente,
208             fecha,
209             importe,
210             estado
211         )
212         VALUES (
213             :idPedido,
214             :idCliente,
215             :fecha,
216             :importeInicial,
217             :estado
218         )
219     };
220
221     verificarFilas(
222         ejecucion,
223         1,
224         "Insertar cabecera"
225     );
226 }
227
228 private BigDecimal insertarLineasYCalcularTotal(
229     DatosPedido datos)
230     throws SQLException {
231
232     BigDecimal importeTotal =
233         BigDecimal.ZERO;
234
235     int idPedido = datos.getIdPedido();
236
237     for (Linea linea : datos.getLineas()) {
238         int idProducto =
239             linea.getIdProducto();
```

```
240
241     int cantidad =
242         linea.getCantidad();
243
244     BigDecimal precio =
245         buscarPrecio(idProducto);
246
247     BigDecimal subtotal =
248         precio.multiply(
249             BigDecimal.valueOf(cantidad)
250         );
251
252     importeTotal =
253         importeTotal.add(subtotal);
254
255     ExecutionContext ejecucion =
256         new ExecutionContext();
257
258     #sql [contexto, ejecucion] {
259         INSERT INTO linea_pedido (
260             id_pedido,
261             id_producto,
262             cantidad
263         )
264         VALUES (
265             :idPedido,
266             :idProducto,
267             :cantidad
268         )
269     };
270
271     verificarFilas(
272         ejecucion,
273         1,
274         "Insertar línea de pedido"
275     );
276 }
277
278     return importeTotal;
279 }
```

```
280
281     private BigDecimal buscarPrecio(
282         int idProducto)
283         throws SQLException {
284
285         BigDecimal precio = null;
286
287         #sql [contexto] {
288             SELECT precio
289             INTO :precio
290             FROM producto
291             WHERE id_producto = :idProducto
292         };
293
294         if (precio == null) {
295             throw new SQLException(
296                 "El producto no tiene precio: " +
297                 idProducto,
298                 "22004"
299             );
300         }
301
302         if (precio.signum() < 0) {
303             throw new SQLException(
304                 "Precio negativo para el producto: " +
305                 idProducto,
306                 "22003"
307             );
308         }
309
310         return precio;
311     }
312
313     private void actualizarImporte(
314         int idPedido,
315         BigDecimal importeTotal)
316         throws SQLException {
317
318         ExecutionContext ejecucion =
319             new ExecutionContext();
```

```
320
321     #sql [contexto, ejecucion] {
322         UPDATE pedido
323         SET importe = :importeTotal
324         WHERE id_pedido = :idPedido
325     };
326
327     verificarFilas(
328         ejecucion,
329         1,
330         "Actualizar importe"
331     );
332 }
333
334 public List<Detalle> consultarPedido(
335     int idPedido)
336     throws SQLException {
337
338     if (idPedido <= 0) {
339         throw new IllegalArgumentException(
340             "El identificador no es válido"
341         );
342     }
343
344     List<Detalle> resultado =
345         new ArrayList<Detalle>();
346
347     DetallePedidoIterator iterador = null;
348     SQLException errorPrincipal = null;
349
350     try {
351         #sql [contexto] iterador = {
352             SELECT p.id_pedido AS idPedido,
353                 c.nombre AS cliente,
354                 p.fecha AS fecha,
355                 p.estado AS estado,
356                 p.importe AS importeTotal,
357                 pr.id_producto AS idProducto,
358                 pr.nombre AS producto,
359                 pr.precio AS precio,
```

```
360         lp.cantidad AS cantidad,
361         lp.cantidad * pr.precio
362         AS subtotal
363     FROM pedido p
364     INNER JOIN cliente c
365         ON c.id_cliente =
366            p.id_cliente
367     INNER JOIN linea_pedido lp
368         ON lp.id_pedido =
369            p.id_pedido
370     INNER JOIN producto pr
371         ON pr.id_producto =
372            lp.id_producto
373     WHERE p.id_pedido = :idPedido
374     ORDER BY pr.nombre
375 };
376
377 while (iterador.next()) {
378     resultado.add(
379         new Detalle(
380             iterador.idPedido(),
381             iterador.cliente(),
382             iterador.fecha(),
383             iterador.estado(),
384             iterador.importeTotal(),
385             iterador.idProducto(),
386             iterador.producto(),
387             iterador.precio(),
388             iterador.cantidad(),
389             iterador.subtotal()
390         )
391     );
392 }
393
394 return resultado;
395
396 } catch (SQLException exception) {
397     errorPrincipal = exception;
398     throw exception;
399 }
```

```
400         } finally {
401             if (iterador != null) {
402                 try {
403                     iterador.close();
404                 } catch (SQLException closeException) {
405                     if (errorPrincipal != null) {
406                         errorPrincipal.addSuppressed(
407                             closeException
408                         );
409                     } else {
410                         throw closeException;
411                     }
412                 }
413             }
414         }
415     }
416
417     private void verificarFilas(
418         ExecutionContext ejecucion,
419         int esperado,
420         String operacion)
421         throws SQLException {
422
423         int actual =
424             ejecucion.getUpdateCount();
425
426         if (actual != esperado) {
427             throw new SQLException(
428                 operacion +
429                 ": se esperaban " +
430                 esperado +
431                 " filas y se obtuvieron " +
432                 actual
433             );
434         }
435     }
436
437     private void rollbackConservandoError(
438         Throwable errorPrincipal) {
439
```

```

440         try {
441             connection.rollback();
442         } catch (SQLException rollbackException) {
443             errorPrincipal.addSuppressed(
444                 rollbackException
445             );
446         }
447     }
448
449     private void restaurarAutoCommit(
450         boolean autoCommitOriginal,
451         SQLException errorPrincipal)
452         throws SQLException {
453
454         try {
455             connection.setAutoCommit(
456                 autoCommitOriginal
457             );
458         } catch (SQLException restoreException) {
459             if (errorPrincipal != null) {
460                 errorPrincipal.addSuppressed(
461                     restoreException
462                 );
463             } else {
464                 throw restoreException;
465             }
466         }
467     }
468 }

```

8.4. Programa de prueba

```

1  package ejemplo.sqlj.ejercicio;
2
3  import java.math.BigDecimal;
4  import java.sql.Connection;
5  import java.sql.Date;
6  import java.sql.DriverManager;
7  import java.sql.SQLException;
8  import java.util.Arrays;

```

```
9  import java.util.List;
10
11  import sqlj.runtime.ref.DefaultContext;
12
13  import ejemplo.sqlj.ejercicio.DatosPedido.Detalle;
14  import ejemplo.sqlj.ejercicio.DatosPedido.Linea;
15
16  public final class PruebaPedidoSQLJ {
17
18      public static void main(String[] args) {
19
20          Connection connection = null;
21          DefaultContext contexto = null;
22
23          try {
24              connection =
25                  DriverManager.getConnection(
26                      System.getenv("DB_URL"),
27                      System.getenv("DB_USER"),
28                      System.getenv("DB_PASSWORD")
29                  );
30
31              contexto =
32                  new DefaultContext(connection);
33
34              ServicioPedidosSQLJ servicio =
35                  new ServicioPedidosSQLJ(
36                      connection,
37                      contexto
38                  );
39
40              DatosPedido pedido =
41                  new DatosPedido(
42                      300,
43                      1,
44                      Date.valueOf("2026-09-08"),
45                      "PENDIENTE",
46                      Arrays.asList(
47                          new Linea(10, 2),
48                          new Linea(11, 1)
```



```

49         )
50     );
51
52     BigDecimal total =
53         servicio.crearPedido(pedido);
54
55     System.out.println(
56         "Importe total: " + total
57     );
58
59     List<Detalle> detalles =
60         servicio.consultarPedido(300);
61
62     for (Detalle detalle : detalles) {
63         System.out.println(
64             detalle.getProducto() +
65             " | cantidad: " +
66             detalle.getCantidad() +
67             " | subtotal: " +
68             detalle.getSubtotal()
69         );
70     }
71
72     } catch (SQLException exception) {
73         registrar(exception);
74
75     } finally {
76         cerrar(contexto, connection);
77     }
78 }
79
80 private static void registrar(
81     SQLException exception) {
82
83     SQLException actual = exception;
84
85     while (actual != null) {
86         System.err.println(
87             "SQLState=" +
88             actual.getSQLState() +

```

```
89         ", código=" +
90         actual.getErrorCode() +
91         ", mensaje=" +
92         actual.getMessage()
93     );
94
95     for (Throwable suppressed :
96         actual.getSuppressed()) {
97
98         System.err.println(
99             "Error secundario: " +
100             suppressed.getMessage()
101         );
102     }
103
104     actual =
105         actual.getNextException();
106 }
107 }
108
109 private static void cerrar(
110     DefaultContext contexto,
111     Connection connection) {
112
113     if (contexto != null) {
114         try {
115             contexto.close();
116         } catch (SQLException exception) {
117             registrar(exception);
118         }
119     }
120
121     if (connection != null) {
122         try {
123             if (!connection.isClosed()) {
124                 connection.close();
125             }
126         } catch (SQLException exception) {
127             registrar(exception);
128         }
129     }
```

```
129         }  
130     }  
131 }
```

9. Errores frecuentes del ejercicio final

9.1. Ejecutar COMMIT dentro del bucle

Incorrecto:

```
1  for (Linea linea : lineas) {  
2      insertarLinea(linea);  
3      connection.commit();  
4  }
```

Si falla la tercera línea, las dos primeras ya están confirmadas.

9.2. Calcular importes con double

Incorrecto:

```
1  double subtotal =  
2      precio * cantidad;
```

Correcto:

```
1  BigDecimal subtotal =  
2      precio.multiply(  
3          BigDecimal.valueOf(cantidad)  
4      );
```

9.3. No comprobar productos repetidos

Si la clave primaria de LINEA_PEDIDO es:

```
1  (id_pedido, id_producto)
```

intentar insertar dos veces el mismo producto producirá una violación de clave.

9.4. Confiar solo en validaciones Java

Aunque Java compruebe el cliente, la base de datos debe conservar la clave ajena.

Otro proceso podría alterar los datos entre la validación y la inserción.

9.5. Silenciar el error de rollback

Incorrecto:

```
1  catch (SQLException exception) {
2      try {
3          connection.rollback();
4      } catch (SQLException ignored) {
5      }
6
7      throw exception;
8  }
```

Correcto:

```
1  catch (SQLException exception) {
2      try {
3          connection.rollback();
4      } catch (SQLException rollbackException) {
5          exception.addSuppressed(
6              rollbackException
7          );
8      }
9
10     throw exception;
11 }
```

9.6. No verificar el número de filas afectadas

Una actualización de cero filas no equivale necesariamente a una operación correcta.

9.7. Devolver al pool una conexión inconsistente

Si fallan:

- COMMIT.
- ROLLBACK.
- Restauración de autocommit.
- Comunicaciones con la base de datos.

La conexión debería invalidarse o cerrarse, no reutilizarse sin más.

10. Variantes para ampliar el ejercicio

1. Aplicar descuentos por categoría.
2. Validar existencia de stock.
3. Bloquear productos durante la reserva.
4. Permitir varias líneas del mismo producto y agruparlas.
5. Añadir impuestos.
6. Guardar el precio aplicado dentro de LINEA_PEDIDO.
7. Añadir una tabla de histórico de estados.
8. Introducir control optimista de versión.
9. Añadir cancelación del pedido.
10. Implementar reintento controlado ante bloqueo.
11. Crear el mismo caso con JDBC.
12. Comparar los planes SQL de las consultas.
13. Procesar múltiples pedidos en lote.
14. Incorporar un SAVEPOINT específico del flujo transaccional.
15. Añadir paginación al detalle histórico del cliente.

Guardar el precio aplicado en la línea suele ser importante en sistemas reales, ya que el precio actual del producto podría cambiar después de crear el pedido.

11. Ejercicios adicionales con solución

11.1. Nivel básico: contar clientes de una ciudad

Enunciado

Recuperar el número de clientes de una ciudad recibida como parámetro.

Solución

```
1 public long contarClientes(  
2     DefaultContext contexto,  
3     String ciudad)  
4     throws SQLException {  
5  
6     if (ciudad == null ||  
7         ciudad.trim().isEmpty()) {  
8
```

```

9         throw new IllegalArgumentException(
10             "La ciudad es obligatoria"
11         );
12     }
13
14     long numeroClientes = 0;
15
16     #sql [contexto] {
17         SELECT COUNT(*)
18         INTO :numeroClientes
19         FROM cliente
20         WHERE ciudad = :ciudad
21     };
22
23     return numeroClientes;
24 }

```

11.2. Nivel básico: insertar un producto

Enunciado

Insertar un producto con nombre, precio y categoría.

Solución

```

1 public void insertarProducto(
2     DefaultContext contexto,
3     int idProducto,
4     String nombre,
5     BigDecimal precio,
6     String categoria)
7     throws SQLException {
8
9     if (idProducto <= 0) {
10         throw new IllegalArgumentException(
11             "Identificador no válido"
12         );
13     }
14
15     if (precio == null ||
16         precio.signum() < 0) {
17

```

```

18         throw new IllegalArgumentException(
19             "Precio no válido"
20         );
21     }
22
23     ExecutionContext ejecucion =
24         new ExecutionContext();
25
26     #sql [contexto, ejecucion] {
27         INSERT INTO producto (
28             id_producto,
29             nombre,
30             precio,
31             categoria
32         )
33         VALUES (
34             :idProducto,
35             :nombre,
36             :precio,
37             :categoria
38         )
39     };
40
41     if (ejecucion.getUpdateCount() != 1) {
42         throw new SQLException(
43             "No se insertó exactamente un producto"
44         );
45     }
46 }

```

11.3. Nivel intermedio: clientes sin pedidos

Enunciado

Recuperar todos los clientes que no tengan pedidos.

Iterador

```

1 #sql iterator ClienteSinPedidoIterator(
2     int idCliente,
3     String nombre,
4     String email,

```

```
5     String ciudad
6 );
```

Solución

```
1  public List<Cliente> buscarClientesSinPedidos(
2      DefaultContext contexto)
3      throws SQLException {
4
5      List<Cliente> resultado =
6          new ArrayList<Cliente>();
7
8      ClienteSinPedidoIterator iterador = null;
9
10     try {
11         #sql [contexto] iterador = {
12             SELECT c.id_cliente AS idCliente,
13                 c.nombre AS nombre,
14                 c.email AS email,
15                 c.ciudad AS ciudad
16             FROM cliente c
17             WHERE NOT EXISTS (
18                 SELECT 1
19                 FROM pedido p
20                 WHERE p.id_cliente =
21                     c.id_cliente
22             )
23             ORDER BY c.nombre
24         };
25
26         while (iterador.next()) {
27             resultado.add(
28                 new Cliente(
29                     iterador.idCliente(),
30                     iterador.nombre(),
31                     iterador.email(),
32                     iterador.ciudad()
33                 )
34             );
35         }
36     }
```



```
37         return resultado;
38
39     } finally {
40         if (iterador != null) {
41             iterador.close();
42         }
43     }
44 }
```

11.4. Nivel intermedio: productos nunca vendidos

Enunciado

Recuperar productos que no aparezcan en ninguna línea de pedido.

SQL

```
1  SELECT pr.id_producto,
2         pr.nombre,
3         pr.precio,
4         pr.categoria
5  FROM producto pr
6  WHERE NOT EXISTS (
7      SELECT 1
8      FROM linea_pedido lp
9      WHERE lp.id_producto = pr.id_producto
10 );
```

Idea principal

NOT EXISTS expresa la ausencia de una relación y evita los problemas que puede presentar NOT IN si la subconsulta contiene NULL.

11.5. Nivel avanzado: clientes con gasto superior a la media

Enunciado

Calcular el gasto total de cada cliente y recuperar únicamente aquellos cuyo gasto supera la media de gasto por cliente.

SQL

```
1  SELECT c.id_cliente,
2         c.nombre,
```

```

3      SUM(p.importe) AS importe_total
4  FROM cliente c
5  INNER JOIN pedido p
6      ON p.id_cliente = c.id_cliente
7  GROUP BY c.id_cliente, c.nombre
8  HAVING SUM(p.importe) > (
9      SELECT AVG(t.total_cliente)
10     FROM (
11         SELECT SUM(p2.importe) AS total_cliente
12         FROM pedido p2
13         GROUP BY p2.id_cliente
14     ) t
15 );

```

La subconsulta derivada en FROM y sus reglas de alias pueden variar entre bases de datos. Debe probarse en el dialecto elegido.

11.6. Nivel avanzado: actualización con control optimista

Enunciado

Cambiar un pedido de PENDIENTE a ENVIADO sin sobrescribir un cambio concurrente.

Solución

```

1  public void enviarPedido(
2      DefaultContext contexto,
3      int idPedido)
4      throws SQLException {
5
6      String estadoAnterior = "PENDIENTE";
7      String estadoNuevo = "ENVIADO";
8
9      ExecutionContext ejecucion =
10         new ExecutionContext();
11
12     #sql [contexto, ejecucion] {
13         UPDATE pedido
14         SET estado = :estadoNuevo
15         WHERE id_pedido = :idPedido
16             AND estado = :estadoAnterior
17     };

```

```

18
19     int filas =
20         ejecucion.getUpdateCount();
21
22     if (filas == 0) {
23         throw new SQLException(
24             "El pedido no existe o ya no está pendiente"
25         );
26     }
27
28     if (filas != 1) {
29         throw new SQLException(
30             "Número inesperado de pedidos actualizados: " +
31             filas
32         );
33     }
34 }

```

12. Ejercicios avanzados con cursores SQLJ

En SQLJ, los iteradores desempeñan el papel de cursores tipados para recorrer resultados de múltiples filas.

12.1. Cursor nombrado: pedidos pendientes

Enunciado

Recorrer todos los pedidos pendientes de un cliente.

Declaración

```

1  #sql iterator PedidoPendienteIterator(
2      int idPedido,
3      java.sql.Date fecha,
4      BigDecimal importe
5  );

```

Solución

```

1  public void mostrarPedidosPendientes(
2      DefaultContext contexto,

```

```

3      int idCliente)
4      throws SQLException {
5
6      PedidoPendienteIterator iterador = null;
7
8      try {
9          #sql [contexto] iterador = {
10             SELECT id_pedido AS idPedido,
11                 fecha AS fecha,
12                 importe AS importe
13             FROM pedido
14             WHERE id_cliente = :idCliente
15                 AND estado = 'PENDIENTE'
16             ORDER BY fecha, id_pedido
17         };
18
19         while (iterador.next()) {
20             System.out.println(
21                 iterador.idPedido() +
22                 " | " +
23                 iterador.fecha() +
24                 " | " +
25                 iterador.importe()
26             );
27         }
28
29     } finally {
30         if (iterador != null) {
31             iterador.close();
32         }
33     }
34 }

```

12.2. Cursor posicional: productos por categoría

Enunciado

Utilizar un iterador posicional para recorrer los productos de una categoría.

Declaración

```

1  #sql iterator ProductoPosicional(

```

```
2     int,  
3     String,  
4     BigDecimal  
5 );
```

Solución

```
1  public void mostrarProductos(  
2      DefaultContext contexto,  
3      String categoria)  
4      throws SQLException {  
5  
6      ProductoPosicional iterador = null;  
7  
8      try {  
9          #sql [contexto] iterador = {  
10             SELECT id_producto,  
11                 nombre,  
12                 precio  
13             FROM producto  
14             WHERE categoria = :categoria  
15             ORDER BY nombre  
16         };  
17  
18         while (true) {  
19             int idProducto = 0;  
20             String nombre = null;  
21             BigDecimal precio = null;  
22  
23             #sql {  
24                 FETCH :iterador  
25                 INTO :idProducto,  
26                     :nombre,  
27                     :precio  
28             };  
29  
30             if (iterador.endFetch()) {  
31                 break;  
32             }  
33  
34             System.out.println(  

```

```

35         idProducto +
36         " | " +
37         nombre +
38         " | " +
39         precio
40     );
41 }
42
43 } finally {
44     if (iterador != null) {
45         iterador.close();
46     }
47 }
48 }

```

La forma exacta de comprobar el final de un iterador posicional debe contrastarse con la implementación SQLJ concreta.

12.3. Cursor con actualización por fila

Enunciado

Recorrer pedidos pendientes antiguos y marcarlos como REVISAR.

Advertencia

Los cursores actualizables o las operaciones posicionadas requieren declaraciones y capacidades que pueden depender de la implementación. IBM documenta iteradores para operaciones posicionadas mediante `sqlj.runtime.ForUpdate`. [\[public.dhe.ibm.com\]](http://public.dhe.ibm.com)

Declaración orientativa

```

1  #sql public iterator PedidoActualizable
2  implements sqlj.runtime.ForUpdate (
3      int idPedido,
4      String estado
5  );

```

Alternativa portable y clara

En lugar de modificar mediante posición, recuperar los identificadores y ejecutar un UPDATE parametrizado:

```

1  #sql iterator PedidoAntiguoIterator(
2      int idPedido

```

```

3 );
1 public int marcarPedidosParaRevision(
2     DefaultContext contexto,
3     java.sql.Date fechaLimite)
4     throws SQLException {
5
6     PedidoAntiguoIterator iterador = null;
7     int actualizados = 0;
8
9     try {
10         #sql [contexto] iterador = {
11             SELECT id_pedido AS idPedido
12             FROM pedido
13             WHERE estado = 'PENDIENTE'
14                 AND fecha < :fechaLimite
15             ORDER BY id_pedido
16         };
17
18         while (iterador.next()) {
19             int idPedido =
20                 iterador.idPedido();
21
22             String estadoAnterior =
23                 "PENDIENTE";
24
25             String estadoNuevo =
26                 "REVISAR";
27
28             ExecutionContext ejecucion =
29                 new ExecutionContext();
30
31             #sql [contexto, ejecucion] {
32                 UPDATE pedido
33                 SET estado = :estadoNuevo
34                 WHERE id_pedido = :idPedido
35                     AND estado = :estadoAnterior
36             };
37
38             actualizados +=
39                 ejecucion.getUpdateCount();

```

```
40         }
41
42         return actualizados;
43
44     } finally {
45         if (iterador != null) {
46             iterador.close();
47         }
48     }
49 }
```

Para un volumen grande, una única sentencia sería normalmente más eficiente:

```
1  UPDATE pedido
2  SET estado = 'REVISAR'
3  WHERE estado = 'PENDIENTE'
4  AND fecha < ?;
```

El procesamiento fila a fila solo está justificado si cada fila necesita una lógica individual.

13. Lista final de decisiones prácticas

Antes de escribir la consulta

- ¿La estructura SQL es estable?
- ¿La consulta puede devolver una o varias filas?
- ¿Alguna columna puede contener NULL?
- ¿Se necesita SQL dinámico?
- ¿La sintaxis es SQL estándar?
- ¿Existe alguna dependencia del fabricante?

Para consultas

- Utilizar SELECT INTO solo para una fila.
- Utilizar iteradores para varias filas.
- Seleccionar solo las columnas necesarias.
- Incluir ORDER BY si el orden importa.
- Cerrar siempre el iterador.
- Controlar el volumen del resultado.

Para modificaciones

- Incluir un filtro seguro en UPDATE y DELETE.

- Verificar el número de filas afectadas.
- No confirmar parcialmente.
- Ejecutar rollback ante errores.
- Mantener las restricciones en la base de datos.
- Tratar correctamente errores de integridad.

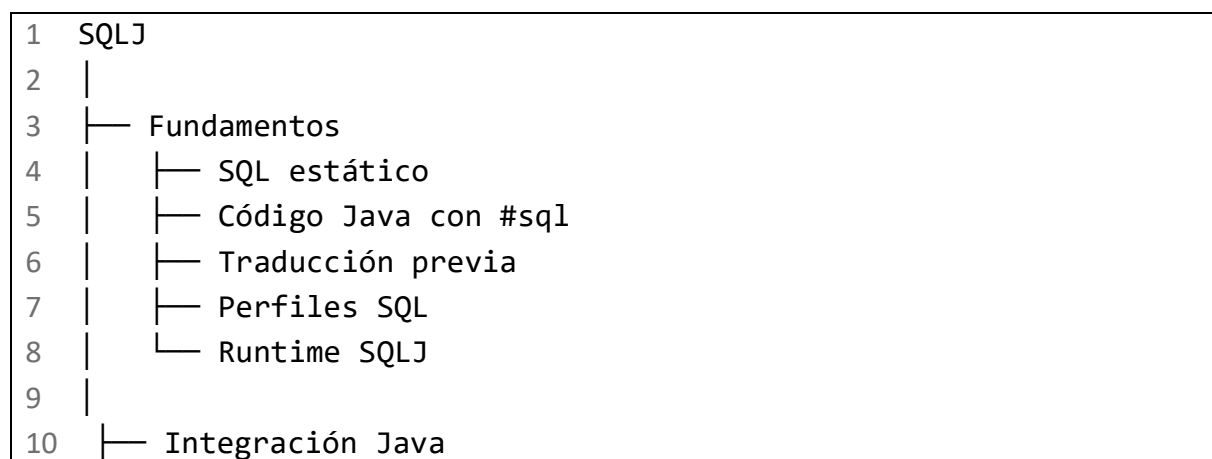
Para transacciones

- Definir la unidad de negocio.
- Desactivar autocommit cuando corresponda.
- Utilizar la misma conexión en el contexto y en el control transaccional.
- Ejecutar COMMIT una sola vez al completar la operación.
- Ejecutar ROLLBACK ante cualquier fallo.
- Conservar errores secundarios.
- Restaurar el estado de la conexión.
- Invalidar conexiones de estado incierto.

Para mantenimiento

- Documentar extensiones Oracle o Db2.
 - No editar código generado por el traductor.
 - Mantener sincronizados iteradores y consultas.
 - Probar la traducción contra un esquema representativo.
 - Incluir pruebas de cero, una y varias filas.
 - Probar errores de restricciones.
 - Probar rollback y errores de commit.
 - Revisar planes de ejecución e índices.
-

14. Mapa conceptual final



11			— Variables host
12			— Entrada
13			— Salida
14			
15			— Contextos
16			— DefaultContext
17			— Contexto declarado
18			
19			— ExecutionContext
20			— Estado de ejecución
21			— Filas afectadas
22			
23			— SQLException
24			— SQLState
25			— Código del fabricante
26			— Excepciones enlazadas
27			— Excepciones suprimidas
28			
29			— Consultas
30			— SELECT INTO
31			— Una fila
32			
33			— Iteradores
34			— Nombrados
35			— Posicionales
36			— Recorrido
37			— Cierre
38			
39			— JOIN
40			— INNER JOIN
41			— LEFT JOIN
42			— RIGHT JOIN
43			— Varias tablas
44			
45			— Subconsultas
46			— EXISTS
47			— NOT EXISTS
48			— IN
49			— NOT IN
50			— UNION

51			└─ UNION ALL
52			
53			└─ Modificación
54			└─ INSERT
55			└─ UPDATE
56			└─ Control de concurrencia
57			└─ DELETE
58			└─ Integridad referencial
59			
60			└─ Procesamiento
61			└─ GROUP BY
62			└─ HAVING
63			└─ ORDER BY
64			└─ Funciones agregadas
65			└─ COUNT
66			└─ SUM
67			└─ AVG
68			└─ MIN
69			└─ MAX
70			
71			└─ Valores NULL
72			└─ IS NULL
73			└─ IS NOT NULL
74			└─ COALESCE
75			└─ Tipos Java anulables
76			
77			└─ Transacciones
78			└─ Autocommit
79			└─ Unidad de negocio
80			└─ COMMIT
81			└─ ROLLBACK
82			└─ Estado de la conexión
83			└─ Tratamiento de errores
84			
85			└─ Comparación
86			└─ SQLJ
87			└─ SQL estático
88			└─ Menos código
89			└─ Traducción
90			└─ Iteradores tipados

91			
92			JDBC
93			SQL dinámico
94			PreparedStatement
95			ResultSet
96			Control detallado
97			
98			Portabilidad
99			SQL estándar
100			Dialecto de base de datos
101			Traductor SQLJ
102			Driver JDBC
103			Oracle
104			IBM Db2

15. Comprobación final de conocimientos

Al finalizar las cuatro entregas, el alumno debería ser capaz de explicar y aplicar:

- Qué es SQLJ y qué problema resuelve.
- Qué significa SQL estático.
- Cómo funciona el traductor SQLJ.
- Qué artefactos puede generar el proceso de traducción.
- Cómo se compila y ejecuta una aplicación SQLJ.
- Cómo se establece un contexto de conexión.
- Qué diferencia existe entre conexión, contexto de conexión y contexto de ejecución.
- Cómo se envían parámetros mediante variables host.
- Cómo se recuperan valores mediante SELECT INTO.
- Qué ocurre con cero, una o varias filas.
- Cómo se declaran iteradores nombrados y posicionales.
- Cómo se recorren y cierran los iteradores.
- Cómo ejecutar INSERT, UPDATE y DELETE.
- Cómo comprobar las filas afectadas.
- Cómo utilizar INNER JOIN, LEFT JOIN y RIGHT JOIN.
- Cómo combinar tres o más tablas.
- Cómo utilizar UNION y UNION ALL.
- Cómo escribir subconsultas.
- Cómo utilizar EXISTS, NOT EXISTS, IN y NOT IN.
- Cómo aplicar GROUP BY y HAVING.

- Cómo utilizar las funciones de agregación.
- Cómo tratar valores NULL.
- Cómo ordenar resultados.
- Cómo delimitar una transacción.
- Cuándo ejecutar COMMIT.
- Cuándo ejecutar ROLLBACK.
- Cómo gestionar fallos durante el rollback o el commit.
- Cómo procesar SQLException.
- Cómo interpretar SQLState y el código del fabricante.
- Cómo evitar modificaciones masivas accidentales.
- Cómo proteger las consultas mediante parámetros.
- Cómo seleccionar los tipos Java adecuados.
- Cómo diferenciar SQLJ de JDBC.
- Cuándo conviene conservar SQLJ.
- Cuándo conviene utilizar JDBC.
- Cómo combinar SQLJ y JDBC dentro de una misma aplicación.
- Qué construcciones deben verificarse para Oracle, Db2 u otra implementación.

Conclusión

SQLJ proporciona una integración concisa y tipada de SQL estático dentro de Java. Su ventaja principal aparece cuando las consultas son estables y existe un proceso SQLJ consolidado que aprovecha la traducción, los iteradores y la validación anticipada.

JDBC proporciona más flexibilidad, mayor control directo y mejor adaptación a SQL dinámico. En sistemas existentes, ambas tecnologías pueden coexistir: SQLJ para operaciones estables y JDBC para búsquedas o sentencias cuya estructura cambia durante la ejecución. La decisión debe basarse en la arquitectura, el soporte del fabricante, las capacidades del equipo y el coste real de mantenimiento, no únicamente en la cantidad de código.